

校园跑腿系统 MVC 接口与模块开发规范

1. 文档信息

项目	内容
项目名称	校园中转分发与跑腿服务管理系统
开发模式	ASP.NET Core MVC 全栈模块开发
后端语言	C#
数据库	Oracle 19c
页面技术	Razor View / Bootstrap
数据访问	Controller -> Service -> Repository -> OracleDbHelper -> Oracle
适用分工	校园跑腿系统_从零开始10人分工方案

2. 文档目的

本文档按新版从零开始分工方案编写。项目采用 ASP.NET Core MVC 开发，各模块负责人需要同时完成 Controller、Service、Repository、ViewModel、View 和基本测试。

本文档用于统一：

- MVC 分层边界。
- 各模块 Controller 路由。
- 页面入口和跳转关系。
- 模块对应数据库表。
- 中文枚举值。
- 最小演示闭环。
- 模块验收标准。

3. 最小演示闭环

项目优先跑通以下主流程：

管理员初始化基础数据
普通用户注册 / 登录
普通用户维护地址
普通用户发布任务
普通用户模拟支付
跑腿员申请并通过审核
跑腿员进入任务大厅接单
跑腿员更新任务状态
普通用户确认签收
普通用户评价或投诉
管理员处理退款 / 投诉
管理员查看结算、审计和统计报表

如果时间紧，最低可答辩版本必须跑通：

登录 -> 发布任务 -> 支付 -> 接单 -> 更新状态 -> 签收 -> 查看数据库变化

4. 统一 MVC 分层规范

后端统一采用：

Controller -> Service -> Repository -> OracleDbHelper -> Oracle

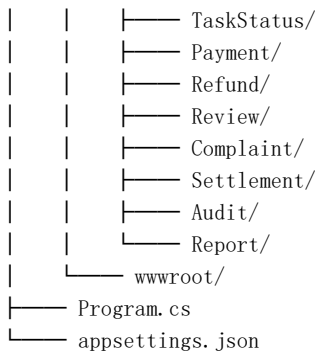
层级	职责
Controller	接收请求、绑定 ViewModel、调用 Service、返回页面或跳转
Service	处理业务规则、状态判断、权限判断和事务流程
Repository	编写 SQL，访问 Oracle 数据库
OracleDbHelper	统一数据库连接、命令执行、事务辅助
ViewModel	页面表单参数和展示数据
View	页面展示，不直接访问数据库

禁止事项：

- 不允许每个人重新创建一个 MVC 项目。
- 不允许绕过 Service 直接在 Controller 中写复杂 SQL。
- 不允许在 View 中访问数据库。
- 不允许随意修改其他人的模块。
- 不允许把中文枚举改成英文值。
- 不允许随意改动表结构。
- 不允许直接提交无法编译的代码。
- 不允许把 Oracle 原始异常直接显示给用户。

5. 推荐目录结构

```
backend/src/CampusDelivery.Api/  
├── Controllers/  
├── Services/  
├── Repositories/  
├── Models/  
├── Persistence/  
├── Presentation/  
│   ├── ViewModels/  
│   ├── Views/  
│   │   ├── Shared/  
│   │   ├── Home/  
│   │   ├── Auth/  
│   │   ├── User/  
│   │   ├── Address/  
│   │   ├── Node/  
│   │   ├── ServiceType/  
│   │   ├── ServiceNodeRule/  
│   │   ├── Runner/  
│   │   ├── Task/  
│   │   └── Assign/
```



6. 公共框架接口

负责人：组员1

功能	Controller / Action	建议路由	页面
首页	HomeController.Index	GET /	Presentation/Views/Home/Index.cshtml
仪表盘	HomeController.Dashboard	GET /Home/Dashboard	Presentation/Views/Home/Dashboard.cshtml
健康检查	HealthController.Get	GET /api/health	JSON
数据库连通检查	DbTestController.Ping	GET /api/DbTest/ping	JSON
错误页	HomeController.Error	GET /Home/Error	Presentation/Views/Shared/Error.cshtml

公共要求：

- 统一 Layout、顶部栏、侧边栏和角色菜单。
- 登录后按角色显示普通用户、跑腿员、管理员菜单。
- 公共页面不得直接访问数据库。
- 数据库连接串统一放在配置文件中。
- 合并后必须 `dotnet build` 通过。

7. 数据库与基础数据规范

负责人：组员2

文件	说明
database/oracle/001_schema.sql	24 张表建表脚本
database/oracle/002_init_base_data.sql	管理员、节点、服务类型、规则等基础数据
database/oracle/003_init_test_data.sql	演示用测试数据
docs/database_dictionary.md	数据库字段说明
docs/chinese_enum_values.md	中文枚举说明

重点数据：

- 管理员账号。
- 普通用户账号。
- 跑腿员账号。

- 校内节点。
- 服务类型。
- 服务节点规则。
- 示例地址。
- 示例任务。
- 示例支付记录。
- 示例评价投诉。

8. 账户、权限、地址模块

负责人：组员3

负责表：

users
user_addresses

8.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
注册页	AuthController.Register	/Auth/Register	GET	Presentation/Views/Auth/Register.cshtml
提交注册	AuthController.Register	/Auth/Register	POST	成功跳转登录
登录页	AuthController.Login	/Auth/Login	GET	Presentation/Views/Auth/Login.cshtml
提交登录	AuthController.Login	/Auth/Login	POST	成功跳转首页
退出登录	AuthController.Logout	/Auth/Logout	POST	跳转登录
个人信息	UserController.Profile	/User/Profile	GET	Presentation/Views/User/Profile.cshtml
修改信息	UserController.Edit	/User/Edit	GET/POST	Presentation/Views/User/Edit.cshtml
地址列表	AddressController.Index	/Address	GET	Presentation/Views/Address/Index.cshtml
新增地址	AddressController.Create	/Address/Create	GET/POST	Presentation/Views/Address/Create.cshtml
修改地址	AddressController.Edit	/Address/Edit/{id}	GET/POST	Presentation/Views/Address/Edit.cshtml
删除地址	AddressController.Delete	/Address/Delete/{id}	POST	跳转地址列表
默认地址	AddressController.SetDefault	/Address/SetDefault/{id}	POST	跳转地址列表

8.2 文件要求

Controllers/AuthController.cs
Controllers/UserController.cs
Controllers/AddressController.cs
Services/UserService.cs
Services/AddressService.cs
Repositories/UserRepository.cs
Repositories/AddressRepository.cs
Presentation/ViewModels/LoginViewModel.cs
Presentation/ViewModels/RegisterViewModel.cs
Presentation/ViewModels/UserViewModel.cs
Presentation/ViewModels/AddressViewModel.cs
Presentation/Views/Auth/*
Presentation/Views/User/*
Presentation/Views/Address/*

8.3 验收标准

- 普通用户可以注册。
- 用户可以登录和退出。
- 禁用账号不能登录。
- 用户可以维护多个地址。
- 同一用户只能有一个默认地址。
- `users` 和 `user_addresses` 表变化正确。

9. 基础资料与跑腿员模块

负责人：组员4

负责表：

nodes
service_types
service_node_rules
runners

9.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
节点列表	<code>NodeController.Index</code>	<code>/Node</code>	GET	<code>Presentation/Views/Node/Index.cshtml</code>
新增节点	<code>NodeController.Create</code>	<code>/Node/Create</code>	GET/POST	<code>Presentation/Views/Node/Create.cshtml</code>
修改节点	<code>NodeController.Edit</code>	<code>/Node/Edit/{id}</code>	GET/POST	<code>Presentation/Views/Node/Edit.cshtml</code>
停用节点	<code>NodeController.Disable</code>	<code>/Node/Disable/{id}</code>	POST	跳转节点列表
服务类型列表	<code>ServiceTypeController.Index</code>	<code>/ServiceType</code>	GET	<code>Presentation/Views/ServiceType/Index.cshtml</code>

功能	Controller / Action	建议路由	请求方式	页面
新增服务类型	<code>ServiceTypeController.Create</code>	<code>/ServiceType/Create</code>	GET/POST	<code>Presentation/Views/ServiceType/Create.cshtml</code>
修改服务类型	<code>ServiceTypeController.Edit</code>	<code>/ServiceType/Edit/{id}</code>	GET/POST	<code>Presentation/Views/ServiceType/Edit.cshtml</code>
服务节点规则	<code>ServiceNodeRuleController.Index</code>	<code>/ServiceNodeRule</code>	GET	<code>Presentation/Views/ServiceNodeRule/Index.cshtml</code>
新增规则	<code>ServiceNodeRuleController.Create</code>	<code>/ServiceNodeRule/Create</code>	GET/POST	<code>Presentation/Views/ServiceNodeRule/Create.cshtml</code>
跑腿员申请	<code>RunnerController.Apply</code>	<code>/Runner/Apply</code>	GET/POST	<code>Presentation/Views/Runner/Apply.cshtml</code>
我的跑腿员信息	<code>RunnerController.MyProfile</code>	<code>/Runner/MyProfile</code>	GET	<code>Presentation/Views/Runner/MyProfile.cshtml</code>
跑腿员审核列表	<code>RunnerController.Pending</code>	<code>/Runner/Pending</code>	GET	<code>Presentation/Views/Runner/Pending.cshtml</code>
审核通过	<code>RunnerController.Approve</code>	<code>/Runner/Approve/{id}</code>	POST	跳转审核列表
审核拒绝	<code>RunnerController.Reject</code>	<code>/Runner/Reject/{id}</code>	POST	跳转审核列表
工作状态维护	<code>RunnerController.UpdateWorkStatus</code>	<code>/Runner/UpdateWorkStatus</code>	POST	跳转个人信息

9.2 文件要求

Controllers/NodeController.cs
 Controllers/ServiceTypeController.cs
 Controllers/ServiceNodeRuleController.cs
 Controllers/RunnerController.cs
 Services/NodeService.cs
 Services/ServiceTypeService.cs
 Services/ServiceNodeRuleService.cs
 Services/RunnerService.cs
 Repositories/NodeRepository.cs
 Repositories/ServiceTypeRepository.cs
 Repositories/ServiceNodeRuleRepository.cs
 Repositories/RunnerRepository.cs
 Presentation/Views/Node/*
 Presentation/Views/ServiceType/*
 Presentation/Views/ServiceNodeRule/*
 Presentation/Views/Runner/*

9.3 验收标准

- 管理员可以维护节点。
- 管理员可以维护服务类型。

- 管理员可以绑定服务类型和节点。
- 普通用户可以申请成为跑腿员。
- 管理员可以审核跑腿员。
- 审核通过后跑腿员可以参与接单流程。

10. 任务发布模块

负责人：组员5

负责表：

```
tasks
food_delivery_details
express_pickup_details
private_task_details
service_node_rules
```

10.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
发布入口	<code>TaskController.Create</code>	<code>/Task/Create</code>	GET	<code>Presentation/Views/Task/Create.cshtml</code>
外卖分发任务	<code>TaskController.CreateFood</code>	<code>/Task/CreateFood</code>	GET/POST	<code>Presentation/Views/Task/CreateFood.cshtml</code>
快递代取任务	<code>TaskController.CreateExpress</code>	<code>/Task/CreateExpress</code>	GET/POST	<code>Presentation/Views/Task/CreateExpress.cshtml</code>
私人跑腿任务	<code>TaskController.CreatePrivate</code>	<code>/Task/CreatePrivate</code>	GET/POST	<code>Presentation/Views/Task/CreatePrivate.cshtml</code>
我的任务	<code>TaskController.MyTasks</code>	<code>/Task/MyTasks</code>	GET	<code>Presentation/Views/Task/MyTasks.cshtml</code>
任务详情	<code>TaskController.Details</code>	<code>/Task/Details/{id}</code>	GET	<code>Presentation/Views/Task/Details.cshtml</code>
取消任务	<code>TaskController.Cancel</code>	<code>/Task/Cancel/{id}</code>	POST	跳转我的任务
全部任务	<code>TaskController.AdminIndex</code>	<code>/Task/AdminIndex</code>	GET	<code>Presentation/Views/Task/AdminIndex.cshtml</code>

10.2 关键业务逻辑

- 任何任务都必须先写入 `tasks` 主表。
- 不同任务类型写入不同明细表。
- `service_type_id + node_id` 必须存在于 `service_node_rules`。
- 任务发布后初始状态为“已创建”。

- 未支付任务可以取消。
- 已接单任务取消需要走退款或管理员处理。

10.3 验收标准

- 三类任务都能发布。
- 主表和明细表都能正确插入。
- 节点规则不匹配时禁止发布。
- 我的任务能按当前用户显示。
- 任务详情能显示对应明细。
- 数据库变化可验证。

11. 接单派单与状态流转模块

负责人：组员6

负责表：

```
assign_records
task_status_logs
tasks
runners
```

11.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
任务大厅	<code>AssignController.Hall</code>	<code>/Assign/Hall</code>	GET	<code>Presentation/Views/Assign/Hall.cshtml</code>
跑腿员抢单	<code>AssignController.Accept</code>	<code>/Assign/Accept/{taskId}</code>	POST	跳转我的接单
我的接单	<code>AssignController.MyAssignments</code>	<code>/Assign/MyAssignments</code>	GET	<code>Presentation/Views/Assign/MyAssignments.cshtml</code>
接单详情	<code>AssignController.Details</code>	<code>/Assign/Details/{id}</code>	GET	<code>Presentation/Views/Assign/Details.cshtml</code>
管理员派单	<code>AssignController.AdminAssign</code>	<code>/Assign/AdminAssign/{taskId}</code>	GET/POST	<code>Presentation/Views/Assign/AdminAssign.cshtml</code>
管理员重派	<code>AssignController.Reassign</code>	<code>/Assign/Reassign/{recordId}</code>	GET/POST	<code>Presentation/Views/Assign/Reassign.cshtml</code>
状态更新页	<code>TaskStatusController.Update</code>	<code>/TaskStatus/Update/{recordId}</code>	GET	<code>Presentation/Views/TaskStatus/Update.cshtml</code>
提交状态更新	<code>TaskStatusController.Update</code>	<code>/TaskStatus/Update/{recordId}</code>	POST	跳转接单详情

功能	Controller / Action	建议路由	请求方式	页面
用户确认签收	TaskStatusController.ConfirmReceipt	/TaskStatus/ConfirmReceipt/{taskId}	POST	跳转任务详情
状态历史	TaskStatusController.History	/TaskStatus/History/{taskId}	GET	Presentation/Views/TaskStatus/History.cshtml

11.2 状态流转

已支付 -> 待接单 -> 已接单 -> 已取件 -> 配送中 -> 待签收 -> 已完成

11.3 验收标准

- 跑腿员可以看到可接任务。
- 抢单成功后生成接单记录。
- 同一任务不能被重复抢单。
- 每次状态变化都有日志。
- 用户确认签收后任务变为“已完成”。
- 跑腿员工作状态变化正确。

12. 支付退款模块

负责人：组员7

负责表：

payments
refunds
tasks
assign_records

12.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
支付页面	PaymentController.Create	/Payment/Create/{taskId}	GET	Presentation/Views/Payment/Create.cshtml
提交支付	PaymentController.Create	/Payment/Create/{taskId}	POST	跳转任务详情
我的支付记录	PaymentController.MyPayments	/Payment/MyPayments	GET	Presentation/Views/Payment/MyPayments.cshtml
支付详情	PaymentController.Details	/Payment/Details/{id}	GET	Presentation/Views/Payment/Details.cshtml
退款申请页	RefundController.Apply	/Refund/Apply/{paymentId}	GET	Presentation/Views/Refund/Apply.cshtml
提交退款申请	RefundController.Apply	/Refund/Apply/{paymentId}	POST	跳转支付记录

功能	Controller / Action	建议路由	请求方式	页面
退款审核列表	<code>RefundController.AdminIndex</code>	<code>/Refund/AdminIndex</code>	GET	<code>Presentation/Views/Refund/AdminIndex.cshtml</code>
退款通过	<code>RefundController.Approve</code>	<code>/Refund/Approve/{id}</code>	POST	跳转审核列表
退款拒绝	<code>RefundController.Reject</code>	<code>/Refund/Reject/{id}</code>	POST	跳转审核列表

12.2 关键业务逻辑

- 未支付任务才允许支付。
- 支付成功后任务进入“已支付”或“待接单”。
- 已取消、已退款任务不能重复支付。
- 退款申请必须关联支付记录。
- 退款审核通过后支付状态应变为“已退款”。
- 退款审核要保留处理结果和原因。

12.3 验收标准

- 用户可以模拟支付。
- 支付后 `payments` 表新增记录。
- 支付后任务状态正确变化。
- 用户可以申请退款。
- 管理员可以审核退款。
- 退款通过后支付状态正确变化。

13. 评价投诉与信誉模块

负责人：组员8

负责表：

reviews
complaints
assign_records
runners

13.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
评价页面	<code>ReviewController.Create</code>	<code>/Review/Create/{recordId}</code>	GET	<code>Presentation/Views/Review/Create.cshtml</code>
提交评价	<code>ReviewController.Create</code>	<code>/Review/Create/{recordId}</code>	POST	跳转任务详情
评价列表	<code>ReviewController.Index</code>	<code>/Review</code>	GET	<code>Presentation/Views/Review/Index.cshtml</code>

功能	Controller / Action	建议路由	请求方式	页面
投诉页面	ComplaintController.Create	/Complaint/Create/{recordId}	GET	Presentation/Views/Complaint/Create.cshtml
提交投诉	ComplaintController.Create	/Complaint/Create/{recordId}	POST	跳转投诉列表
我的投诉	ComplaintController.MyComplaints	/Complaint/MyComplaints	GET	Presentation/Views/Complaint/MyComplaints.cshtml
投诉处理列表	ComplaintController.AdminIndex	/Complaint/AdminIndex	GET	Presentation/Views/Complaint/AdminIndex.cshtml
投诉处理	ComplaintController.Handle	/Complaint/Handle/{id}	GET/POST	Presentation/Views/Complaint/Handle.cshtml

13.2 关键业务逻辑

- 只有已完成任务才能评价。
- 一个接单记录最多一条评价。
- 投诉必须关联接单记录。
- 投诉处理结果应保存。
- 投诉成立时影响跑腿员信誉分。
- 信誉分不能低于系统允许范围。

13.3 验收标准

- 用户可以评价已完成任务。
- 重复评价被阻止。
- 用户可以提交投诉。
- 管理员可以处理投诉。
- 投诉成立后跑腿员信誉分变化。
- 数据库记录完整。

14. 结算审计报表模块

负责人：组员9

负责表：

```

settlements
settlement_payment_items
audit_logs
audit_status_log_checks
audit_payment_checks
audit_refund_checks
reports

```

14.1 Controller 路由

功能	Controller / Action	建议路由	请求方式	页面
结算列表	SettlementController.Index	/Settlement	GET	Presentation/Views/Settlement/Index.cshtml
生成结算	SettlementController.Generate	/Settlement/Generate	POST	跳转结算列表
结算详情	SettlementController.Details	/Settlement/Details/{id}	GET	Presentation/Views/Settlement/Details.cshtml
审计日志	AuditController.Index	/Audit	GET	Presentation/Views/Audit/Index.cshtml
审计详情	AuditController.Details	/Audit/Details/{id}	GET	Presentation/Views/Audit/Details.cshtml
报表首页	ReportController.Index	/Report	GET	Presentation/Views/Report/Index.cshtml
生成报表	ReportController.Generate	/Report/Generate	POST	跳转报表首页
订单统计	ReportController.OrderSummary	/Report/OrderSummary	GET	Presentation/Views/Report/OrderSummary.cshtml
支付统计	ReportController.PaymentSummary	/Report/PaymentSummary	GET	Presentation/Views/Report/PaymentSummary.cshtml
投诉统计	ReportController.ComplaintSummary	/Report/ComplaintSummary	GET	Presentation/Views/Report/ComplaintSummary.cshtml

14.2 关键业务逻辑

- 已完成且已支付的任务才进入结算。
- 同一支付记录不能重复结算。
- 有未处理投诉的任务暂不结算。
- 报表可以先使用表格展示，不强制做复杂图表。
- 审计功能优先做查询和异常标记。

14.3 验收标准

- 可以生成跑腿员结算记录。
- 可以查看结算明细。
- 重复结算被阻止。
- 可以查看审计日志。

- 可以查看统计报表。
- 管理员端能展示核心统计结果。

15. 中文枚举总表

所有业务状态、按钮、提示信息尽量使用中文。数据库中的业务状态值也必须使用中文枚举。

类型	中文值
用户角色	普通用户、跑腿员、管理员
账号状态	正常、禁用
是否标志	是、否
节点状态	正常、关闭
服务类型状态	启用、禁用
跑腿员审核状态	待审核、已通过、已拒绝
跑腿员工作状态	空闲、忙碌、离线
任务状态	已创建、已支付、待接单、已接单、已取件、配送中、待签收、已完成、已取消、退款中
接派类型	自主接单、管理员派单、重新派单
支付方式	微信、支付宝、现金
支付状态	未支付、已支付、支付失败、已退款
退款状态	待审核、已通过、已拒绝
投诉状态	待处理、已成立、已驳回

禁止写成：

USER
 RUNNER
 ADMIN
 NORMAL
 DISABLED
 CREATED
 PAID
 WAITING
 Y
 N

16. 模块完成标准

每个模块完成时，至少满足：

- 页面能打开。
- 表单能提交。
- 数据能写入或查询。
- 数据库对应表变化正确。
- SQL Developer 可以验证。
- 角色权限正确。

- 中文枚举正确。
- 错误提示友好。
- `dotnet build` 通过。
- 至少提供一张功能截图。
- 能讲清自己负责的表和业务逻辑。

17. 优先级建议

优先级	内容
P0	项目能运行、数据库能连、登录能进系统
P0	普通用户发布任务
P0	支付后任务可被接单
P0	跑腿员抢单和状态更新
P0	用户确认签收
P1	评价、投诉、退款
P1	管理员审核跑腿员
P1	节点、服务类型、规则管理
P2	结算、审计、报表
P2	页面美化、统计图、复杂筛选

18. Git 分支建议

分支	负责人	用途
<code>main</code>	组员1	稳定可演示版本
<code>dev</code>	组员1	日常合并版本
<code>feature/base-db</code>	组员2	数据库脚本和测试数据
<code>feature/account-address</code>	组员3	账户、权限、地址
<code>feature/base-runner</code>	组员4	基础资料、跑腿员
<code>feature/task</code>	组员5	任务发布
<code>feature/assign-status</code>	组员6	接单派单、状态流转
<code>feature/payment-refund</code>	组员7	支付退款
<code>feature/review-complaint</code>	组员8	评价投诉
<code>feature/settlement-report</code>	组员9	结算审计报表
<code>feature/test-docs</code>	组员10	测试文档答辩

提交信息建议：

完成用户登录功能
完成地址管理页面
完成任务发布主单插入
修复抢单重复判断

补充支付测试数据
整理答辩演示截图